

数学建模 微分方程 6.4 6.5 6.6 题报告

2351114 大数据 朱俊泽

6.4 题目

6.4 根据经验,当一种新商品投入市场后,随着人们对它的拥有量的增加,其销售量 $s(t)$ 下降的速度与 $s(t)$ 成正比。广告宣传可给销量添加一个增长速度,它与广告费 $a(t)$ 成正比,但广告只能影响这种商品在市场上尚未饱和的部分(设饱和量为 M)。建立一个销售 $s(t)$ 的模型。若广告宣传只进行有限时间 τ ,且广告费为常数 a ,问 $s(t)$ 如何变化?

6.4.1 建模

题目描述了一种新商品在市场上的销售动态,并给出了以下信息:

销售量的自然衰减:销售量 $s(t)$ 的下降速度与当前的销售量 $s(t)$ 成正比,比例常数为 λ ($\lambda > 0$)。

广告的促进作用:广告可以增加销售量,其增长速度与广告费 $a(t)$ 成正比。但广告只能影响市场上尚未饱和的部分,即 $(1-s(t)/M)$,其中 M 是市场的饱和量。

广告策略:广告只在有限时间 τ 内进行,且广告费为常数 a 。具体形式为:

$$a(t) = \begin{cases} a/\tau, & 0 < t < \tau, \\ 0, & t \geq \tau. \end{cases}$$

目标是建立一个描述销售量 $s(t)$ 随时间变化的模型,并求解 $s(t)$ 的具体表达式。

根据题意,销售量的变化率 $\frac{ds}{dt}$ 由两部分组成:一是广告的促进作用 $pa(t)(1 - \frac{s(t)}{M})$,其中 p 是预设的广告的促进系数。还有自然衰减作用 $-\lambda s(t)$ 。因此我们假设微分方程为:

$$\frac{ds}{dt} = pa(t) \left(1 - \frac{s(t)}{M} \right) - \lambda s(t)$$

其中广告策略如下:

$$a(t) = \begin{cases} a/\tau, & 0 < t < \tau, \\ 0, & t \geq \tau. \end{cases}$$

在第一阶段广告投放阶段 $0 < t < \tau$: $a(t) = a/\tau$ 。方程表示为:

$$\frac{ds}{dt} + \left(\lambda + \frac{pa}{M\tau} \right) s = \frac{pa}{\tau}$$

我们整理方程，令：

$$b = \lambda + \frac{pa}{M\tau}, c = \frac{pa}{\tau}$$

则有：

$$\frac{ds}{dt} + bs = c.$$

这是一个一阶的线性常微分方程，那么我们使用通解：

$$s(t) = \frac{c}{b} + Ce^{-bt}.$$

带入初始条件 $s(0) = s_0$ ，解为 $C = s_0 - \frac{c}{b}$ 。带入方程有：

$$s(t) = \frac{c}{b}(1 - e^{-bt}) + s_0 e^{-bt}.$$

第二阶段广告停止 $t \geq \tau$ ， $a(t)=0$ 。方程表示为：

$$\frac{ds}{dt} = -\lambda s.$$

此时解为：

$$s(t) = s(\tau)e^{-\lambda(t-\tau)}.$$

其中 $s(\tau)$ 是广告投放阶段在 $t=\tau$ 时的值：

$$s(\tau) = \frac{c}{b}(1 - e^{-b\tau}) + s_0 e^{-b\tau}.$$

综合两个阶段， $s(t)$ 的完整表达式为：

$$s(t) = \begin{cases} \frac{c}{b}(1 - e^{-bt}) + s_0 e^{-bt}, & 0 < t \leq \tau \\ s(\tau)e^{-\lambda(t-\tau)}, & t \geq \tau. \end{cases}$$

6.4.2 代码

本题目是非解决的题目，不用提供代码

6.4.3 结果

本题目是非解决的题目，不用提供最终结果

6.5 题目

6.5 用龙格库塔方法求微分方程数值解,画出解的图形,对结果进行分析比较。

(1) $x^2 y'' + xy' + (x^2 - n^2)y = 0, y\left(\frac{\pi}{2}\right) = 2, y'\left(\frac{\pi}{2}\right) = -\frac{2}{\pi}$, 为 Bessel 方程, $n = \frac{1}{2}$ 时, 精确解 $y = \sin x \sqrt{\frac{2\pi}{x}}$ 。

(2) $y'' + y \cos x = 0, y(0) = 1, y'(0) = 0$, 幂级数解

$$y = 1 - \frac{1}{2!}x^2 + \frac{2}{4!}x^4 - \frac{9}{6!}x^6 + \frac{55}{8!}x^8 - \dots$$

6.5.1 建模

(1): 第一小问分析:

做数值变换将二阶方程变换为一阶方程组

$$\begin{cases} y_1' = y_2 \\ y_2' = \left(\frac{n^2}{x^2} - 1\right)y_1 - \frac{y_2}{x} \end{cases}$$

初始条件:

$$y_1\left(\frac{\pi}{2}\right) = 2, y_2\left(\frac{\pi}{2}\right) = -\frac{2}{\pi}$$

最后利用龙格库塔法求解

(2): 第二小问分析:

同样先做数值变换将二阶方程化为一阶方程组

$$\begin{cases} y_1' = y_2 \\ y_2' = -y_1 \cos x \end{cases}$$

初始条件:

$$y_1(0) = 1, y_2(0) = 0.$$

最后利用龙格库塔法求解

6.5.2 代码

6-5-1.py

```
from scipy.integrate import solve_ivp
import numpy as np
import matplotlib.pyplot as plt

def bessel_equations(x, y, n=0.5):
    y1, y2 = y
    dy1dx = y2
```

```

    dy2dx = (n**2 / x**2 - 1) * y1 - y2 / x
    return [dy1dx, dy2dx]
# Exact solution
def exact_solution(x):
    return np.sin(x) * np.sqrt(2 * np.pi / x)
# Avoid x=0 due to singularity, start slightly above
x0 = 0.1
# Initial conditions adjusted for starting at x0
y0 = [exact_solution(x0),
      np.sqrt(2*np.pi/x0)*np.cos(x0) - np.sin(x0)*np.sqrt(2*np.pi/x0**3)/2]
# Numerical solution from x0 to 8
sol = solve_ivp(bessel_equations, [x0, 8], y0, t_eval=np.linspace(x0, 8, 1000))
# Create plot
plt.figure(figsize=(10, 6))
x_plot = np.linspace(0.1, 8, 1000) # Start slightly above 0
# Numerical solution (from solve_ivp)
plt.plot(sol.t, sol.y[0], 'b-', linewidth=2, label='Numerical Solution')
# Exact solution
plt.plot(x_plot, exact_solution(x_plot), 'r--', linewidth=2, label='Exact
Solution')
plt.xlabel('x', fontsize=12)
plt.ylabel('y(x)', fontsize=12)
plt.title('Solution of Bessel Equation (n=1/2) from 0 to 8', fontsize=14)
plt.legend(fontsize=12)
plt.grid(True, linestyle='--', alpha=0.7)
plt.xlim(0, 8) # Ensure we show from 0 to 8
# Highlight the singularity at x=0
plt.annotate('Singularity at x=0', xy=(0.2, 2), xytext=(1, 3),
            arrowprops=dict(facecolor='black', shrink=0.05))
plt.show()

```

6-5-1-plus.py

```

from scipy.integrate import solve_ivp
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.patches import Rectangle

# ===== Exact Solution =====
def exact_solution(x):
    """Exact solution for Bessel equation (n=0.5)"""
    return np.sin(x) * np.sqrt(2 * np.pi / x)
def exact_derivative(x):
    """Derivative of exact solution (for initial conditions)"""
    return np.sqrt(2*np.pi/x)*np.cos(x) - np.sin(x)*np.sqrt(2*np.pi/x**3)/2
# ===== Numerical Solution =====

```

```

def bessel_equations(x, y, n=0.5):
    """First-order system for Bessel equation"""
    y1, y2 = y
    dy1dx = y2
    dy2dx = (n**2 / x**2 - 1) * y1 - y2 / x
    return [dy1dx, dy2dx]

# Avoid singularity at x=0
x0 = 0.1
x_end = 8
y0 = [exact_solution(x0), exact_derivative(x0)]
# Numerical solution with high accuracy
sol = solve_ivp(bessel_equations, [x0, x_end], y0,
                t_eval=np.linspace(x0, x_end, 1000),
                method='DOP853', rtol=1e-8, atol=1e-8)

# ===== Analysis =====
x_vals = sol.t
y_num = sol.y[0]
y_exact = exact_solution(x_vals)
# 1. Absolute error
abs_error = np.abs(y_num - y_exact)
# 2. Relative error (with safeguard against division by zero)
relative_error = np.abs(y_num - y_exact) / (np.abs(y_exact) + 1e-12)
# 3. Residual analysis
def residual(x, y, y_prime, y_double_prime):
    """Compute residual of Bessel equation"""
    return x**2 * y_double_prime + x * y_prime + (x**2 - 0.25) * y

# Numerical derivatives
y_prime_num = np.gradient(y_num, x_vals)
y_double_prime_num = np.gradient(y_prime_num, x_vals)
residuals = residual(x_vals, y_num, y_prime_num, y_double_prime_num)

# ===== Plotting =====
plt.figure(figsize=(14, 10))
# ----- Main plot: Solutions -----
plt.subplot(2, 2, (1, 2))
plt.plot(x_vals, y_num, 'b-', lw=2, label='Numerical Solution')
plt.plot(x_vals, y_exact, 'r--', lw=1.5, label='Exact Solution')
plt.xlabel('x')
plt.ylabel('y(x)')
plt.title('Bessel Equation Solution Comparison (n=0.5)')
plt.legend()
plt.grid(True)
# Add zoom area indicator
zoom_x = (3, 4)
zoom_rect = Rectangle((zoom_x[0], -0.5), zoom_x[1]-zoom_x[0], 1,

```

```

        linewidth=1, edgecolor='k', facecolor='none', linestyle='--')
plt.gca().add_patch(zoom_rect)
plt.text(zoom_x[0], -0.6, 'Zoom Area', ha='left')
# ----- Subplot 1: Absolute Error -----
plt.subplot(2, 2, 3)
plt.semilogy(x_vals, abs_error, 'g-', lw=1.5)
plt.xlabel('x')
plt.ylabel('Absolute Error (log scale)')
plt.title('Absolute Error Analysis')
plt.grid(True, which='both')
# ----- Subplot 2: Zoomed Comparison -----
plt.subplot(2, 2, 4)
zoom_mask = (x_vals >= zoom_x[0]) & (x_vals <= zoom_x[1])
plt.plot(x_vals[zoom_mask], y_num[zoom_mask], 'b-', lw=2, label='Numerical')
plt.plot(x_vals[zoom_mask], y_exact[zoom_mask], 'r--', lw=1.5, label='Exact')
plt.xlabel('x')
plt.ylabel('y(x)')
plt.title(f'Zoomed View ({zoom_x[0]} < x < {zoom_x[1]})')
plt.legend()
plt.grid(True)
plt.tight_layout()
# ===== Statistics =====
print("\n===== Error Statistics =====")
print(f"Max Absolute Error: {np.max(abs_error):.2e}")
print(f"Mean Absolute Error: {np.mean(abs_error):.2e}")
print(f"Max Relative Error: {np.max(relative_error)*100:.2f}%")
print(f"Mean Relative Error: {np.mean(relative_error)*100:.2f}%")
print(f"Max Residual: {np.max(np.abs(residuals)):.2e}")
# ===== Residual Plot =====
plt.figure(figsize=(10, 5))
plt.semilogy(x_vals, np.abs(residuals), 'm-', lw=1.5)
plt.xlabel('x')
plt.ylabel('Residual (log scale)')
plt.title('Bessel Equation Residual Analysis')
plt.grid(True, which='both')
plt.show()

```

6-5-2.py

```

from scipy.integrate import solve_ivp
import numpy as np
from math import factorial
import matplotlib.pyplot as plt

def equation(x, y):
    y1, y2 = y

```

```

    dy1dx = y2
    dy2dx = -y1 * np.cos(x)
    return [dy1dx, dy2dx]
# 初始条件和求解区间
sol = solve_ivp(equation, [0, 2], [1, 0], t_eval=np.linspace(0, 2, 1000))
# 幂级数解 (前 5 项)
def series_solution(x):
    return (1 - (1/factorial(2))*x**2 + (2/factorial(4))*x**4 -
            (9/factorial(6))*x**6 + (55/factorial(8))*x**8)

```

6-5-2-plus.py

```

from scipy.integrate import solve_ivp
import numpy as np
from math import factorial
import matplotlib.pyplot as plt
from matplotlib.patches import Rectangle

# ===== Series Solution =====
def series_solution(x):
    """Power series approximation (first 5 terms)"""
    return (1 - (1/factorial(2))*x**2 + (2/factorial(4))*x**4 -
            (9/factorial(6))*x**6 + (55/factorial(8))*x**8)
# ===== Numerical Solution =====
def equation(x, y):
    """First-order system for y'' + y cos(x) = 0"""
    y1, y2 = y
    dy1dx = y2
    dy2dx = -y1 * np.cos(x)
    return [dy1dx, dy2dx]
# Numerical solution with high accuracy
sol = solve_ivp(equation, [0, 2], [1, 0],
                t_eval=np.linspace(0, 2, 1000),
                method='DOP853', rtol=1e-8, atol=1e-8)
# ===== Analysis =====
x_vals = sol.t
y_num = sol.y[0] # Numerical solution
y_series = series_solution(x_vals) # Series approximation
# 1. Absolute error
abs_error = np.abs(y_num - y_series)
# 2. Relative error (with safeguard against division by zero)
relative_error = np.abs(y_num - y_series) / (np.abs(y_num) + 1e-12)
# 3. Residual analysis (for numerical solution)
def residual(x, y, y_prime, y_double_prime):
    """Compute residual of y'' + y cos(x) = 0"""
    return y_double_prime + y * np.cos(x)

```

```

# Numerical derivatives
y_prime_num = np.gradient(y_num, x_vals)
y_double_prime_num = np.gradient(y_prime_num, x_vals)
residuals = residual(x_vals, y_num, y_prime_num, y_double_prime_num)

# ===== Plotting =====
plt.figure(figsize=(14, 10))

# ----- Main plot: Solutions -----
plt.subplot(2, 2, (1, 2))
plt.plot(x_vals, y_num, 'b-', lw=2, label='Numerical Solution')
plt.plot(x_vals, y_series, 'r--', lw=1.5, label='Series Approximation')
plt.xlabel('x')
plt.ylabel('y(x)')
plt.title('Solution Comparison:  $y'' + y \cos(x) = 0$ ')
plt.legend()
plt.grid(True)

# Add zoom area indicator
zoom_x = (1.5, 2.0)
zoom_rect = Rectangle((zoom_x[0], -0.2), zoom_x[1]-zoom_x[0], 0.4,
                      linewidth=1, edgecolor='k', facecolor='none', linestyle='--')

plt.gca().add_patch(zoom_rect)
plt.text(zoom_x[0], -0.25, 'Zoom Area', ha='left')

# ----- Subplot 1: Absolute Error -----
plt.subplot(2, 2, 3)
plt.semilogy(x_vals, abs_error, 'g-', lw=1.5)
plt.xlabel('x')
plt.ylabel('Absolute Error (log scale)')
plt.title('Absolute Error Analysis')
plt.grid(True, which='both')

# ----- Subplot 2: Zoomed Comparison -----
plt.subplot(2, 2, 4)
zoom_mask = (x_vals >= zoom_x[0]) & (x_vals <= zoom_x[1])
plt.plot(x_vals[zoom_mask], y_num[zoom_mask], 'b-', lw=2, label='Numerical')
plt.plot(x_vals[zoom_mask], y_series[zoom_mask], 'r--', lw=1.5, label='Series')
plt.xlabel('x')
plt.ylabel('y(x)')
plt.title(f'Zoomed View ({zoom_x[0]} < x < {zoom_x[1]})')
plt.legend()
plt.grid(True)
plt.tight_layout()

# ===== Statistics =====
print("\n==== Error Statistics =====")
print(f"Max Absolute Error: {np.max(abs_error):.2e}")
print(f"Mean Absolute Error: {np.mean(abs_error):.2e}")
print(f"Max Relative Error: {np.max(relative_error)*100:.2f}%")

```

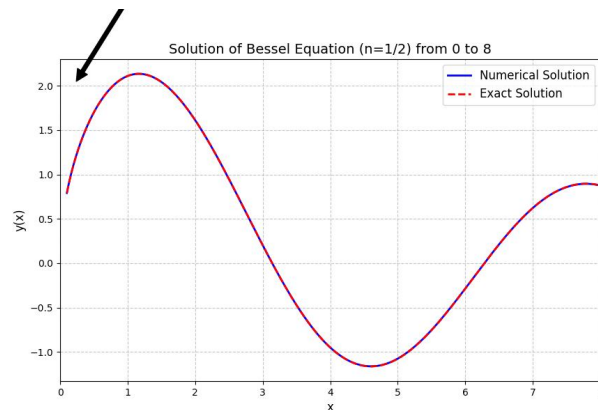


```

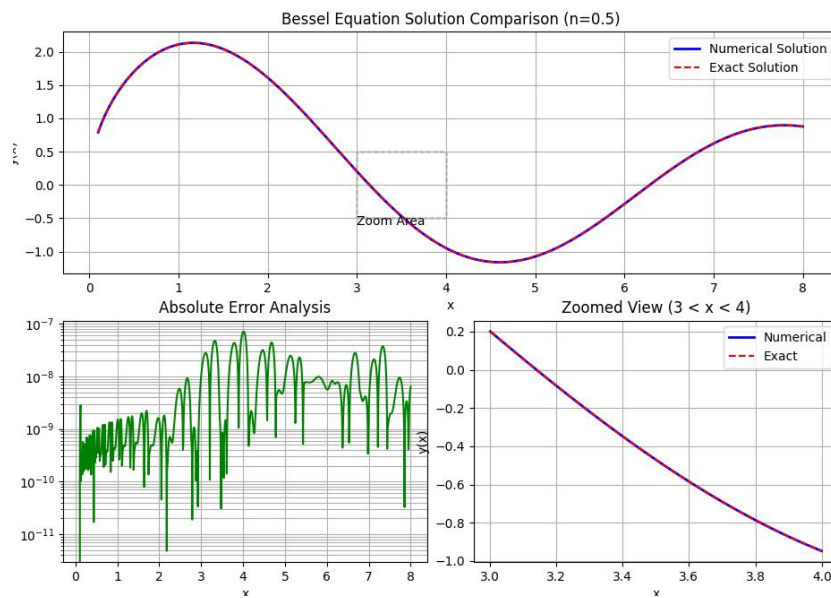
print(f"Mean Relative Error: {np.mean(relative_error)*100:.2f}%")
print(f"Max Residual: {np.max(np.abs(residuals)):.2e}")
# ===== Residual Plot =====
plt.figure(figsize=(10, 5))
plt.semilogy(x_vals, np.abs(residuals), 'm-', lw=1.5)
plt.xlabel('x')
plt.ylabel('Residual (log scale)')
plt.title('Residual Analysis: |y\\\'\' + y cos(x)|')
plt.grid(True, which='both')
plt.show()

```

6.5.3 结果



6-5-1 近似解和精确解图示



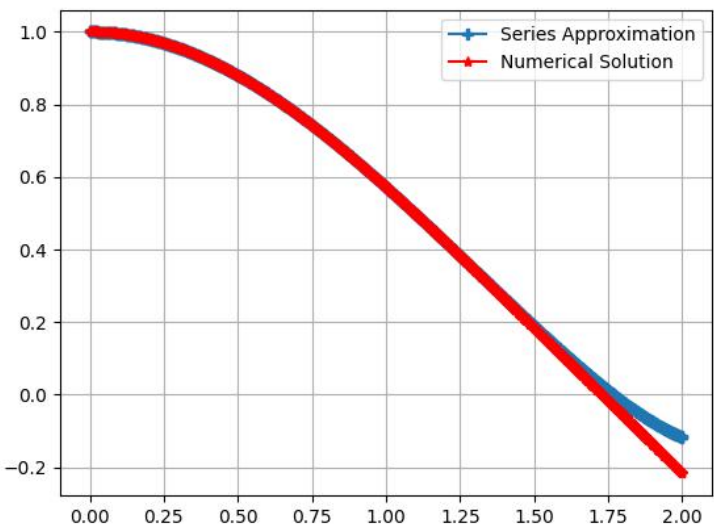
6-5-1 误差分析

最大绝对差异: 7.16e-08

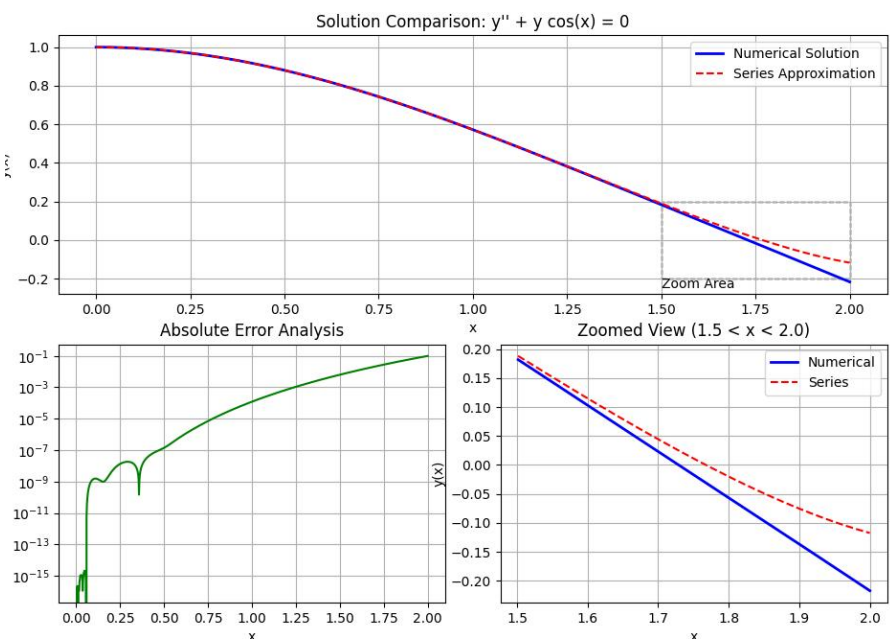
平均绝对差异: 9.35e-09

平均相关差异: 0.00%

最大残差: 2.72e+01



6-5-2 近似解和精确解图示



6-5-2 误差分析

最大绝对差异: 9.97e-02

平均绝对差异: 9.50e-03

平均相关差异: 4.50%

最大残差: 5.00e-01

6.6 题目

6.6 一只小船渡过宽为 d 的河流,目标是起点 A 正对着的另一岸 B 点。已知河水流速 v_1 与船在静水中的速度 v_2 之比为 k 。

(1) 建立小船航线的方程,求其解析解。

(2) 设 $d=100\text{m}$, $v_1=1\text{m/s}$, $v_2=2\text{m/s}$,用数值解法求渡河所需时间、任意时刻小船的位置及航行曲线,作图,并与解析解比较。

6.6.1 建模

小船从点 $B(0,0)$ 出发,目标是点 $A(d,0)$ 。

小船速度 $v_1=1\text{m/s}$, 水流速度 $v_2=2\text{m/s}$ 。

水流速度 v_1 沿 y 轴正方向(向上),即水流速度为 $(0,v_1)$ 。

小船的实际速度是自身速度 v_2 和水流速度 v_1 的矢量和。小船的航向始终指向目标点 $A(d,0)$ 。

题目要求: 建立小船轨迹的微分方程。求小船的轨迹方程 $y=y(x)$ 。求小船从 $B(0,0)$ 到 $A(d,0)$ 的航行时间。

小艇的自身速度 v_2 始终朝向原点 B (方向由当前位置 (x,y) 确定)。

水流速度 v_1 沿 y 轴正方向。

由此,速度分量为:

$$\frac{dx}{dt} = -v_2 \frac{x}{\sqrt{x^2+y^2}}, \quad \frac{dy}{dt} = v_1 - v_2 \frac{y}{\sqrt{x^2+y^2}}.$$

微方程推导:

$$\frac{dy}{dx} = \frac{v_1 - v_2 \frac{y}{\sqrt{x^2+y^2}}}{-v_2 \frac{x}{\sqrt{x^2+y^2}}} = k \frac{\sqrt{x^2+y^2}}{x} + \frac{y}{x}$$

其中 $k = v_1/v_2$

该方程为齐次方程,通过变量替换 $u = \frac{y}{x}$ 可化简为可分离变量形式。

变量替换:

$$u = \frac{y}{x}$$

带入微分方程化解可得:

$$x \frac{du}{dx} = -k\sqrt{1+u^2}.$$

分离变量并积分，结合初值条件 $x = d, u = 0$ ，解得：

$$u = \frac{1}{2} \left[\left(\frac{x}{d} \right)^{-k} - \left(\frac{x}{d} \right)^k \right].$$

最终解析解： 代回 $y = ux$ ，得轨迹方程：

$$y = \frac{d}{2} \left| \left(\frac{x}{d} \right)^{1-k} - \left(\frac{x}{d} \right)^{1+k} \right|, \quad 0 \leq x \leq d.$$

设小船在时刻 t 的坐标为 $(x(t), y(t))$,其运动满足以下微分方程组：

$$\begin{cases} \frac{dx}{dt} = -\frac{v_2 x}{\sqrt{x^2 + y^2}}, & x(0) = d, \\ \frac{dy}{dt} = v_1 - \frac{v_2 y}{\sqrt{x^2 + y^2}}, & y(0) = 0, \end{cases}$$

其中： v_1 为水流速度(沿 y 轴正方向), v_2 为小船自身速度(始终指向原点 B), d 为初始位置 $x(0) = d$ (即起点到原点 B 的水平距离)。

6.6.2 代码

6-6.py

```
import numpy as np
from scipy.integrate import solve_ivp
import matplotlib.pyplot as plt

# 定义参数
d = 100
v1 = 1
v2 = 2
k = v1 / v2
# 定义解析解函数
y_analytic = lambda x: d / 2 * ((x/d)**(1 - k) - (x/d)**(1 + k))
# 绘制解析解曲线
x_vals = np.linspace(0, d, 400)
plt.plot(x_vals, y_analytic(x_vals), label='Series Approximation')
# 定义微分方程的右端项
def dxy(t, xy):
```

```

    x, y = xy
    denominator = np.sqrt(x**2 + y**2) + 1e-8 # 防止除以零
    dxdt = -v2 * x / denominator
    dydt = v1 - v2 * y / denominator
    return [dxdt, dydt]
# 定义事件函数：当 x < 1e-3 时触发终止
def event_x_zero(t, xy):
    x, y = xy
    return x - 1e-3 # 当 x < 1e-3 时返回负值，触发事件
event_x_zero.terminal = True # 事件触发后终止积分
event_x_zero.direction = -1 # 仅检测下降穿过零点的事件
# 求解微分方程（时间区间设为足够大的范围，如 [0, 1000]）
sol = solve_ivp(
    dxy,
    [0, 1000], # 时间区间上限设为较大值
    [d, 0], # 初始条件
    method='RK45',
    events=event_x_zero, # 绑定事件检测
    dense_output=True
)
# 提取渡河时间
crossing_time = sol.t_events[0][0]
print(f"渡河时间 = {crossing_time:.2f} 秒")
# 绘制数值解
plt.plot(sol.y[0], sol.y[1], 'r*', markersize=3, label='Numerical Solution')
# 图形设置
plt.xlabel('x (米)')
plt.ylabel('y (米)')
plt.gca().invert_xaxis() # 反转 x 轴（从 100 到 0）
plt.legend()
plt.grid(True)
plt.show()

```

6-6-plus.py

```

import numpy as np
from scipy.integrate import solve_ivp
from scipy.interpolate import interp1d
import matplotlib.pyplot as plt
from matplotlib.patches import Rectangle

# ===== 参数优化 =====
d = 100 # 初始水平距离 (m)
v1 = 1 # 水流速度 (m/s)
v2 = 2 # 小船速度 (m/s)
k = v1 / v2 # 速度比

```

```

def analytic_solution(x):
    """小船航迹的解析解公式"""
    return (d / 2) * ((x/d)**(1 - k) - (x/d)**(1 + k))

# ===== 事件检测优化 =====
def event_x_zero(t, xy):
    """当 x < 1m 时终止计算"""
    return xy[0] - 1 # 检测 x=1 时触发终止
event_x_zero.terminal = True
event_x_zero.direction = -1

# ===== 数值解求解优化 =====
def dydt(t, xy):
    x, y = xy
    denominator = np.sqrt(x**2 + y**2) + 1e-6 # 增大 epsilon 提高稳定性
    return [-v2 * x / denominator, v1 - v2 * y / denominator]

# 控制计算规模: 减少时间点数量
sol = solve_ivp(
    dydt,
    [0, 200],
    [d, 0],
    method='DOP853',
    events=event_x_zero, # 添加事件检测
    rtol=1e-6,           # 适当降低精度要求
    atol=1e-6,
    t_eval=np.linspace(0, 200, 2000) # 减少到 2000 个点
)

# ===== 数据对齐优化 =====
# 提取有效数据 (去除事件触发后的无效点)
valid = sol.t <= sol.t_events[0][0]
x_num = sol.y[0][valid]
y_num = sol.y[1][valid]
# 使用线性插值替代三次样条
y_interp = interp1d(
    np.flip(x_num), # 输入必须单调递增
    np.flip(y_num),
    kind='linear',
    bounds_error=False,
    fill_value=np.nan
)

# ===== 误差分析优化 =====
x_eval = np.linspace(1, 100, 500) # 从 x=1 开始避免边界问题
y_numeric = y_interp(x_eval)
valid_mask = ~np.isnan(y_numeric)

# ===== 简化绘图 =====
plt.figure(figsize=(12, 6))

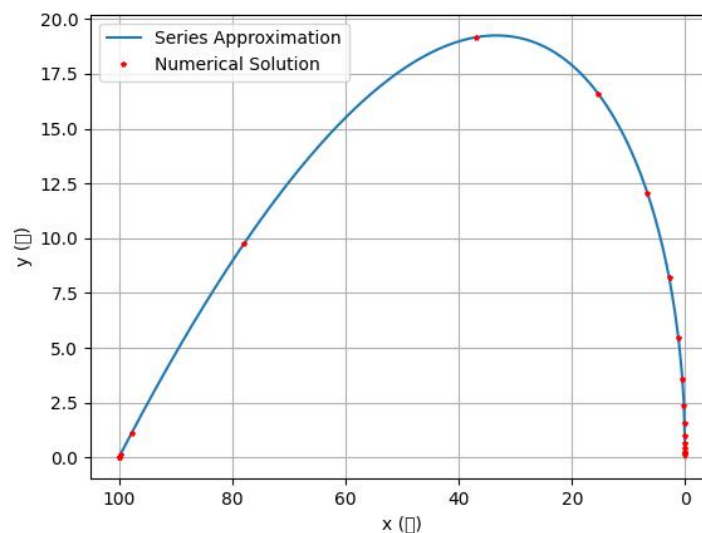
```

```

# 主图
plt.plot(x_eval[valid_mask], analytic_solution(x_eval[valid_mask]),
         'b--', label='解析解')
plt.plot(x_eval[valid_mask], y_numeric[valid_mask],
         'r--', label='数值解')
plt.xlabel('x (米)'), plt.ylabel('y (米)')
plt.title('优化后的小船航线对比')
plt.legend(), plt.grid(True)
# 误差子图
plt.figure(figsize=(12, 6))
abs_error = np.abs(y_numeric[valid_mask] - analytic_solution(x_eval[valid_mask]))
plt.semilogy(x_eval[valid_mask], abs_error, 'g-')
plt.xlabel('x (米)'), plt.ylabel('difference')
plt.title('difference')
plt.grid(True)
plt.show()
# ===== 控制台优先输出 =====
print("\n===== 运行状态报告 =====")
print(f"实际计算时间区间: [0, {sol.t[-1]:.2f}]秒")
print(f"有效数据点数: {len(x_num)}")
print(f"最大绝对误差: {np.nanmax(abs_error):.2e}米")

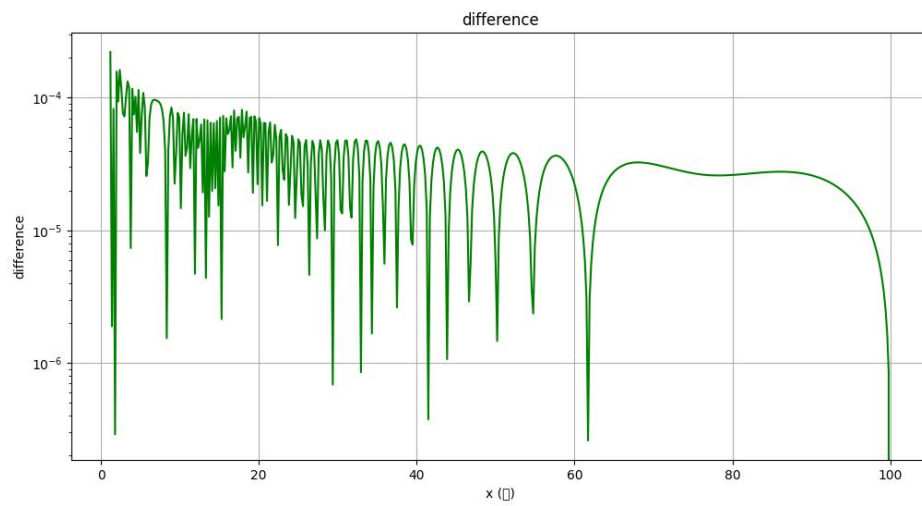
```

6.6.3 结果



渡河小船解析解和数值解图示

渡河时间: 66.50 秒



6-6 误差分析

有效数据点数: 617

最大绝对误差: $2.21\text{e-}04$ 米